מספר תעודת זהות
יש לרשום את כל תשע הספרות

יש להדביק כאן את מדבקת הנבחן

האוניברסיטה הפתוחה

כ"א בתמוז תשפ"ג

מס' שאלון - 524

10

ביולי 2023

סמסטר 2023ב

מס' מועד 84

20905 / 4

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה **10** עמודים

מבנה הבחינה:

בבחינה 3 שאלות.
משך זמן הבחינה 3 שעות.
עליכם לענות על כל השאלות.
משקל השאלות מפורט בגוף השאלון.

שימו לב:
את תשובותיכם רשמו אך ורק במחברת הבחינה.
כתבו בכתב יד ברור ומסודר.

חומר עזר המותר בשימוש:
ספר הקורס "Essentials of Programming Languages".
מדריך הלמידה "שפות תכנות".

חומר עזר:
ספר הקורס: essentials of programming languages
מדריך הלמידה. מותרות הערות בכתב יד, ע"ג הספרים.
אין להכניס חומר מודפס, דיגיטלי או כל חומר אחר מכל סוג שהוא.

בהצלחה !!!

אינכם חייבים
להחזיר את השאלון לאוניברסיטה הפתוחה

שאלה 1 (35 נקודות)

שאלה זו עוסקת בשדרוג של שפת "ויהי" (שפת LET) המתוארת בספר הלימוד בפרק 3 בעמודים 60-74.

<u>תזכורת</u>:

בשפת "ויהי", כאשר מתעוררת שגיאה, מודפסת הודעת שגיאה (ע"י eopl:error) ממנה לא חוזרים, וריצת התוכנית נפסקת. שגיאות יכולות להיות בעת חישוב ביטוי, חיפוש משתנה לא קיים בסביבה, חילוץ סוג ערך שגוי מתוך expval. למשל, בעת ביצוע פעולת חיסור, אם אחד הביטויים אינו מספרי, או בעת ביצוע ביטוי if בו הביטוי הראשון אינו בוליאני. או בביטוי zero? כאשר הביטוי הנבדק בו אינו מספרי, וכדומה.

המקומות בקוד מהם מתעוררות שגיאות:

המחלצים השונים של expval בקובץ data-strucures.

הפרוצדורה apply-environment, כאשר אינה מוצאת את המשתנה בסביבה.

עצם אופן חישוב הביטויים השונים בשפה.

בשאלה זו נרצה לשנות את האופן של טיפול בשגיאות. במקום הדפסת הודעות שגיאה והפסקת ריצת התוכנית, כפי שממומש בספר, נחליף זאת בהטמעת עבודה עם **חריגות** (exceptions) כפי שמקובל במגוון שפות תכנות.

לשם כך יש צורך להטמיע בשפה עבודה עם חריגות, על פי ההסברים הבאים והדוגמאות שלאחריהן.

נוסיף לשפה מספר ביטויים חדשים שיאפשרו למתכנת בשפה זו להריץ בצורה מבוקרת קוד העלול לייצר חריגה, אפשרות לתפיסת חריגות שונות והמשך טיפול בהן , וכן אפשרות לייצר ולזרוק חריגה באופן יזום.

להלן הביטויים החדשים אותם יש לממש:

Expression ::= try { ***Expression*** }
{ catch [***Except***] : ***Expression*** ; }*
{ finally : ***Expression*** ; }*

```
try-exp (exp1 excpts excptexps finexps)
```

Expression ::= throw ***Except***

```
throw-exp (excpt)
```

Except : : = ***message***

```
Exception (msg)
```

message : : = general | not a number | not a boolean | environment

הסבר על המרכיבים החדשים :

Except – מתאר חריגה, ומוגדר להיות ***message***, כאשר ***message*** מוגדר להיות מגוון של הודעות חריגה כפי שמוגדר בדקדוק מעלה.

message – מתאר את החריגות השונות שניתן לזרוק :

חריגה כללית – general

חריגה not a number כאשר מצפים שמשהו יהיה מספר והוא לא כזה.

חריגה not a boolean כאשר מצפים שמשהו יהיה ערך בוליאני והוא לא כזה

חריגת סביבה environment – כאשר מחפשים בסביבה משתנה והוא לא נמצא בה.

ביטוי try-exp - ביטוי זה מורכב ממספר חלקים :

exp1 - מתאר את הביטוי המופיע בבלוק ה-try אותו מעוניינים להריץ ועלול לייצר חריגה.

excpts – רשימת חריגות המופיעים בחלק ה- catch בהם מעוניינים לטפל. החריגה שתטופל היא החריגה הראשונה ברשימה excpts המתאימה לחריגה שנזרקה מביצוע exp1. אם exp1 לא זרק חריגה, או שנזרקה סוג חריגה שלא מופיעה ברשימה excpts אז לא תטופל אף אחת מהחריגות ב-excpts. כמו כן, excpts יכולה להיות גם רשימה ריקה.

excptexps – רשימת ביטויים אחד כנגד כל חריגה ב- excpts שרוצים לטפל בה. הביטויים מתארים מה נדרש לבצע עבור החריגות השונות בהם רוצים לטפל במקרה והן נזרקות מחישוב exp1.

finexps – מתאר את הביטוי המופיע בחלק של finally. החלק של finally הוא אופציונלי, יכול לא להופיע כלל, או להופיע פעם אחת לכל היותר (יש לוודא זאת). לכן finexps היא רשימה ריקה או באורך של 1. הביטוי בחלק ה- finally (אם מופיע) יתבצע בכל מקרה. גם אם בלוק ה-try זרק/לא זרק חריגה. כלומר, תחילה מורץ בלוק ה-try, לאחריו בדיקה וביצוע טיפול בחריגה מתאימה (אם היתה ואם מופיע טיפול עבורה). ובכל מקרה, יבוצע חלק ה- finally במידה והוא נרשם.

ביטוי throw-exp

ביטוי בעזרתו המתכנת בשפה יכול לזרוק חריגה ביוזמתו המתוארת ע"י הפרמטר **excpt**

שימו לב, שבשאלה זו יש כאן סוג תשובה חדשה (חריגה) שיכולה לחזור מחישוב ביטויים.

בשאלה זו, יש לממש את הביטויים והמרכיבים החדשים. לעדכן את הטיפול בכלל הביטויים בשפה כך שבמקרים בעייתיים בחישובם יחזירו חריגה מתאימה.

כמו כן, יש לעדכן את כל המקומות בקוד המדפיסים הודעת שגיאה, ובמקום הדפסת שגיאה יחזירו חריגה מתאימה.

להלן מספר דוגמאות שימוש :

דוגמה 1:

```
> (run "let t= -(6,zero?(9))in 78")
(excp-val (Exception "not a number"))
>
```

בדוגמה זו, ניתן לראות שיש ניסיון לבצע פעולת חיסור עם ערך בוליאני (הביטוי zero?) ועל כן, הוחזרה חריגה מביטוי ה-let ולא 78 כפי שכתוב ב-body שלו

דוגמה 2:

```
> (run "-(t,8)")
(excp-val (Exception "environment"))
>
```

בדוגמה זו, הוחזרה חריגת סביבה, מאחר ו-t אינו קיים בסביבת המפרש

דוגמה 3:

```
> (run "throw general")
(excp-val (Exception "general"))
>
```

דוגמה זו ממחישה שימוש בביטוי throw לזריקת חריגה באופן יזום (במקרה זה חריגה מסוג general)

דוגמה 4:

```
> (run "try
        {
           -(if  -(4,6) then 70 else 20 , 45)
       }
       catch  [general] : 11   ;
       catch  [not a number] : 12   ;
       catch  [not a boolean] : 13 ;
       catch  [environment] :  14 ;
       finally : 200;
       ")
(num-val 13)
> |
```

דוגמה זו, ממחישה ביצוע פעולת חיסור, בין תוצאת ביטוי if לבין 45, חישוב ביטוי ה-if מייצר חריגה, מכיוון שאינו בודק ביטוי בוליאני. ועל כן, תהליך החישוב של פעולת החיסור נכשל ובעצמו זורק את החריגה, ולכן בלוק ה-try זורק במקרה זה חריגת not a boolean, ה- catch השלישי מתאים לסוג חריגה זו, ועל כן הוחזר 13. (לפני החזרת 13 בוצע חלק ה-finally)

דוגמה 5:

```
> (run  "try         {
           -(if  -(4,6) then 70 else 20 , 45)
        }
        catch  [general] : 11   ;
        catch  [not a number] : 12   ;
        catch  [not a boolean] : throw general ;
        catch  [environment] :  14 ;
        finally : 200;
       ")
(excp-val (Exception "general"))
> |
```

כמו בדוגמה הקודמת, נזרקת מבלוק ה-try חריגת not a boolean והיא מתאימה ל- catch השלישי, שמחזיר חריגה של general

המשך הבחינה בעמוד הבא

דוגמה 6:

```
> (run  "try          {
            -(if  -(4,6) then 70 else 20 , 45)
        }
        catch  [general] : 11   ;
        catch  [not a number] : 12   ;
        catch  [environment] :  14 ;
        finally : 200;
       ")
(excp-val (Exception "not a boolean"))
> |
```

בדוגמה זו, בלוק ה- try זורק חריגת not a boolean אין טיפול בסוג חריגה זו, מבוצע חלק ה-finally והחריגה שנזרקה מוחזרת כתוצאה של כלל התוכנית.

ממשו והטמיעו את השינויים הדרושים כפי שהוגדרו והודגמו בתוך שפת **"יהי"** (**שפת LET**).

בתשובתכם, הקפידו להסביר **היכן בדיוק** נדרשים שינויים ותוספות בקבצי המפרש, **ומהם** השינויים והתוספות בקוד המפרש.

הנחיות כלליות לפתרון:

1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
3) הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- terminals, non-terminals ופעולות של סגור (*} { או $^+$} {)

המשך הבחינה בעמוד הבא

שאלה 2 (35 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "**וירמוז**"(IMPLICIT-REFS).

בשאלה זו, נרצה לאפשר עבודה עם משתנים מסוג const, כלומר לאחר קביעת ערכם, הם לא ניתנים לשינוי.

להזכירכם, בשפה זו, משתנים נוצרים ומתווספים לסביבה, כאשר מבצעים ביטוי let ומגדירים במסגרתו משתנה חדש, או כאשר מבצעים ביטוי של הפעלת פרוצדורה, שבו בתהליך ההפעלה מתווסף פרמטר הפרוצדורה לסביבה עמה מחשבים את גוף הפרוצדורה, ובה הוא מקבל את ערכו של הארגומנט הנשלח בביטוי ההפעלה.

נרצה לאפשר למשתמש להוסיף במקרים כאלה את האופציה לציין שמדובר על משתנה שהוא const. כלומר, רק אם המתכנת השתשמש באופציה לציון שהשימוש במשתנה יהיה כ-const אז לא ניתן יהיה לשנות את ערכו יותר. אך אם לא היה שימוש באופציה של const אזי המשתנה ניתן לשינוי.

להלן הדקדוקים החדשים המגדירים מחדש את ביטוי let-exp ואת ביטוי call-exp :

Expression ::= let ***option Identifier*** = ***Expression***
in ***Expression***

```
let-exp (opt id exp1 body)
```

Expression ::= (***Expression option Expression***)

```
call-exp (rator opt rand)
```

option ::= ε | const

<u>הסבר התחביר:</u>

<u>option</u> - מוגדר להיות המילה const או המילה הריקה (פשוט כלום – זה מה שמייצג ε)

<u>ביטוי let-exp החדש</u> - בהגדרת המשתנה id בביטוי זה ניתן לכתוב לפניו את המילה const או לא לכתוב כלום לפני הגדרת המשתנה.

<u>ביטוי call-exp החדש</u> – לפני הארגומנט הנשלח (rand) ניתן לציין את המילה const או לא לציין דבר.

הסבר הסמנטיקה:

בביטויים החדשים, אם option הוא המילה הריקה אזי הביטויים יתפקדו ויחושבו כרגיל (כמו המימוש המקורי)

אם option היא המילה const אזי:

- בביטוי let-exp לא ניתן יהיה לשנות את ערכו של id במסגרת ה-body של ה-let, אם יהיה ניסיון כזה יש להדפיס שגיאה מתאימה
- בביטוי הפעלת פרוצדורה, לא ניתן יהיה לשנות את ערכו של פרמטר הפרוצדורה המופעלת במסגרת ביצוע ה- body שלה, וגם כאן, אם יהיה ניסיון כזה יש להדפיס שגיאה

להלן מספר דוגמאות שימוש להמחשה:

דוגמה 1:

```
> (run "let w= 8
        in
          begin
             set w = 60;
             w
          end")
(num-val 60)
>
```

דוגמה זו ממחישה התנהגות רגילה של ביטוי let ללא שימוש במילה const לפני הגדרת המשתנה w. בתחילה ערכו של w הוא 8, במסגרת ביצוע גוף ה- let ערכו משתנה ל- 60, וכל ביטוי ה-let מחזיר את ערכו של w ולכן הוחזר 60. וכן היה אפשר לשנות את ערכו של w.

דוגמה 2:

```
> (run "let const w=  8
        in
          begin
             set w = 60;
             w
          end")
interp.scm:104:16: assign-exp: cannot assign to const w
>
```

כמו בדוגמה הקודמת, אלא שכאן לפני המשתנה w המוגדר ע"י ביטוי ה- let נרשמה המילה const, ולכן הניסיון לשנותו בגוף ה- let הדפיס שגיאה.

דוגמה 3:

```
> (run  "let p = proc(w)
                 begin
                   set w= 800;
                   w
                 end
      in (p 60)")
(num-val 800)
>
```

בדוגמה זו, מודגמת התנהגות רגילה של הפעלת פרוצדורה p בעלת פרמטר w. הפרוצדורה מופעלת עם ערך 60 עבור w, בגוף הפרודורה ערכו של w משתנה ל- 800 ומוחזר (ולא היתה בעיה לשנותו) ולכן ערכה של כל התוכנית הוא 800.

דוגמה 4:

```
> (run "let p = proc(w)
                begin
                  set w= 800;
                  w
                end
     in (p const 60)")
interp.scm:104:16: assign-exp: cannot assign to const w
>
```

דוגמה זו דומה לדוגמה הקודמת, אלא שכאן בביטוי הפעלת הפרוצדורה נרשמה המילה const לפני הארגומנט הנשלח (60) ולכן ערכו של w (פרמטר הפרוצדורה) הוא 60 ואינו ניתן לשינוי במהלך ביצוע גוף הפרוצדורה, ולכן הניסיון לשנותו בגוף הפרוצדורה גרם לשגיאה.

ממשו והטמיעו את השינויים הדרושים בתוך שפת **"וירמוז"** (**שפת IMPLICIT-REFS**).

בתשובתכם, הקפידו להסביר **היכן בדיוק** נדרשים שינויים ותוספות בקבצי המפרש, **ומהם** השינויים והתוספות.

ייתכנו מספר דרכי פתרון שונים – הסבירו בנוסף בקצרה את הדרך בה בחרתם לממש.

יש לשים לב ולבדוק מקרים של שימוש לא תקין בתחביר. ולהדפיס הודעות שגיאה מתאימות.

הנחיות כלליות לפתרון:

1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
3) הפתרון אמור לשמור על הגדרות השפה המקוריות. (כגון אופן ניהול הסביבה וסוגי הערכים בה, אופן ניהול ההחסן והערכים שניתן לשמור בו, וכדומה)
4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)

5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier

6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- terminals, non-terminals ופעולות של סגור (*}{ או $^+$}{)

שאלה 3 (30 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

(proc (x : ?) zero?(x) let z=7 in z)

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים).לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (15 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב- substitution ו- unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

בהצלחה!